

Part 2 — Trade-offs & Production Readiness Analysis

Credit Decision Data Platform · UAE credit products (personal finance, BNPL, credit-card alternative)

This analysis is grounded in the Part 1 implementation in this repository. Concrete references use the actual file and contract names: docs/SPEC.md (single source of truth), the Glue jobs under glue/silver_layer/ and glue/gold_layer/, the DQ rule definitions in glue/common/dq_rules.py, the identity core in glue/common/text_match.py, and the Terraform in infra/terraform/.

1. Conflict Resolution Trade-offs

How customer-key conflicts are handled (Part 1)

The four sources arrive with four different native keys — AECB on emirates_id, fraud on phone + email, AML on full_name + date_of_birth, and the internal profile on internal_customer_uuid (SPEC §2). glue/silver_layer/build_customer_identity_xref.py fuses them

into one golden record, customer_identity_xref, using a **deterministic-first, probabilistic-fallback** strategy (SPEC §6):

1. **Spine seeding.** PostgreSQL is the identity spine. Each row deterministically seeds one master_customer_id = UUIDv5(internal_customer_uuid) (text_match.master_id), so ids are ****stable** and reproducible** across reruns — critical for idempotent MERGES and for regulators re-deriving the same id years later.
2. **Deterministic joins.** AECB attaches on normalised emirates_id; fraud attaches when **both** normalised phone (E.164) and lowercased email match. A strong-key exact match ⇒ match_confidence = 1.00, match_method = DETERMINISTIC.
3. **Probabilistic fallback.** Partial fraud matches (only phone or email) and **all** AML matches (soundex(full_name) + date_of_birth, fuzzy by construction) go to a weighted scorer: text_match.weighted_match_confidence — Jaro-Winkler on name, exact bonuses on dob/phone/email/eid, **renormalised over the fields actually populated on both sides** so a name+dob-only pair is still scored on a 0–1 scale.
4. **Thresholds (SPEC §11).** ≥ 0.85 attaches; 0.70–0.85 attaches but sets needs_manual_review = TRUE; < 0.70 produces an **UNRESOLVED** record — nothing is ever dropped.
5. **Survivorship.** Conflicting demographics resolve by source priority POSTGRES > AECB > FRAUD > AML, most-recent-timestamp within a priority.

Every decision carries `match_method`, `match_confidence`, `matched_on`, and `needs_manual_review`, so the resolution is itself auditable (surfaced by `athena/views/v_unresolved_identities.sql`).

Key trade-offs made

- **Deterministic-first over ML matching.** Deterministic joins are explainable, cheap, and defensible to a regulator ("these two records share an exact Emirates ID"). The cost is recall: transliteration and data-entry noise (see edge cases) slip past exact joins and land on the fuzzy scorer, which is where precision risk concentrates.
- **A hard 0.85/0.70 threshold over a learned decision boundary.** Simple, tunable, and unit-tested (`tests/test_text_match.py`), but a fixed threshold is blunt — it applies the same bar to a high-value personal-finance application and a AED-500 BNPL, when the cost of a wrong merge differs by orders of magnitude.
- **Preserve-everything (UNRESOLVED) over drop-on-no-match.** Guarantees no source data is silently lost and keeps the pipeline non-blocking, at the cost of an UNRESOLVED backlog that needs a stewardship process to work down.
- **PostgreSQL as the single spine.** One canonical id simplifies everything downstream, but it means a customer who exists at the bureau/fraud/AML providers but **not yet** in the internal profile cannot be resolved — they become UNRESOLVED until the profile CDC catches up.

Edge cases & failure modes

Edge case	What happens today	Risk
Shared phone/email (family members, one household email)	Fraud single-key match → scorer; may over-merge two people onto one master	False merge — one person's fraud signal contaminates another's decision
Emirates ID typo / reuse	Deterministic join misses (typo) or wrongly joins (reissued EID)	Split identity, or cross-person leakage
Arabic name transliteration (Sheikh vs Shaikh, Mohammed/Muhammad/Mohamed)	Jaro-Winkler + soundex usually clears 0.85 (verified in the sample), but exotic variants land in the 0.70–0.85 review band	Manual-review backlog; occasional missed link
Stale contact data (the intentional sample conflict: profile phone ≠ fraud phone, email matches)	Single-key candidate → scorer; POSTGRES wins survivorship	Correct here, but a stale email + stale phone = no match at all
Multiple candidates above threshold	Highest score wins (<code>_best_per_master</code>); ties broken arbitrarily	Non-deterministic tie-breaking; a marginally-higher wrong candidate can win
No spine row yet (CDC lag on customer_profile)	UNRESOLVED sentinel; decision still produced	Decision made against a thin/unresolved identity
Soundex collapses distinct names (soundex is coarse)	Over-blocks; distinct people share a block, scorer must disambiguate on dob alone	False merge when dob also collides
Merge/unmerge after the fact	Not supported — no re-link or split workflow exists	A wrong merge is sticky until manually corrected

The two most dangerous are **false merges** (a decision uses another person's fraud/AML/AECB data — a compliance and credit-risk incident) and **CDC lag** (deciding before the spine exists). The design mitigates false merges with the review band + `matched_on` provenance, but there is no automated

un-merge — that is an accepted 48-hour cut (§5).

What I'd do differently at 10x scale (100K/day → ~3–4M identities)

- **The scorer is the scaling bottleneck.** Today it blocks on `soundex(name)+dob` and `phone/email`; at volume this candidate generation degrades toward $O(n^2)$ within crowded blocks (common Arabic soundex codes are huge). Move to a **purpose-built entity-resolution engine** — AWS Entity Resolution, or Splink/Zingg on Spark — with **LSH/MinHash blocking** and probabilistic linkage trained on labelled pairs, instead of hand-tuned weights.
 - **Incremental resolution, not full recompute.** Resolve only the delta each run against a persisted candidate index; today the xref job reasons over full silver tables.
 - **A stewardship console with a feedback loop.** The `needs_manual_review` and `UNRESOLVED` queues (already materialised as a view) should feed a UI where analysts confirm/reject links, and those labels become training data for the matcher — turning a static threshold into a learning system.
 - **Merge/unmerge as first-class events.** Model identity as an append-only log of link/unlink events so a wrong merge can be reversed without rewriting history, and every decision references the identity *as it was at decision time* (which the immutable snapshot in §3 already partially guarantees).
 - **Per-product risk-weighted thresholds.** Raise the bar for high-limit personal finance, relax it for low-value BNPL, so review effort follows financial exposure.
-

2. Data Quality Strategy

Must-pass vs warning — the justification

The DQ job (`glue/gold_layer/run_dq_scorecard_mart.py`, rules in `glue/common/dq_rules.py`, mirrored in

`dq/soda/`) runs two tiers (SPEC §8). The dividing principle is **legal/financial validity of a single decision vs health of the pipeline as a whole**:

- **MUST-PASS (blocking → quarantine).** A failure means *this decision record is invalid and must not reach the risk mart or influence a credit outcome*:
 - `decision_id / application_id / master_customer_id` non-null & unique — without these the record cannot be joined, audited, or attributed to a person.
 - `fraud_score` $\in [0, 1]$, `aecb_credit_score` $\in [300, 900]$ when present — an out-of-range score signals a corrupt feed; acting on it is worse than not acting.
 - `product_code` in the allowed enum, `decision_timestamp` not in the future / not > 48h stale at load.
- **Identity resolved** (not the `UNRESOLVED` sentinel) — a decision attributed to "UNRESOLVED" has no accountable subject.

These quarantine the offending row (`decision_input_quarantine`, with `dq_fail_reasons`) rather than fail the whole batch — one bad row never blocks 9,999 good ones.

- **WARN (non-blocking → SNS alert, dashboarded).** A breach means *the system is degrading, but any individual passing decision is still valid*:

- `input_completeness_score` ≥ 0.75 for $\geq 95\%$ of rows.
- AML PENDING rate $\leq 5\%$.
- Source freshness within SLA (AECB < 24h, fraud < 1h, AML < 6h, profile CDC < 15m).

Why the split matters: if freshness or completeness were must-pass, a single slow upstream (AECB SFTP running late) would quarantine an entire day of applications and halt lending — a self-inflicted outage. Conversely, making identity-resolution a warning would let unattributed decisions reach production. The rule of thumb encoded in Part 1: ****must-pass protects the integrity of one decision; warn protects the reliability of the platform.****

Balancing decision speed vs data completeness

The platform is explicitly **non-blocking on source availability**. `build_decision_input` LEFT-joins the four sources onto each decision and computes `input_completeness_score` (fraction of expected inputs present, SPEC §5). A decision is produced even with missing inputs; completeness is *measured*, not *required*. This is the deliberate trade-off:

- **Speed side:** BNPL and card-alternative products need sub-second-to-minutes decisions; waiting for a slow bureau pull would blow the SLA and lose the sale. The pipeline emits a decision with whatever is present and records exactly what was missing.
- **Completeness side:** the missing-ness is not hidden — it lowers `input_completeness_score`, can trip the WARN threshold, and (critically) is frozen into the immutable snapshot (§3), so the credit engine's policy can react to a thin file (e.g. lower limit, refer, or step-up) with full knowledge that AECB was absent. ****The platform does not decide the credit outcome — it delivers an honest, complete-as-possible input with its gaps labelled****, and the policy layer owns the speed/completeness call per product.

What happens when AECB data is delayed

Concretely, given the Part 1 design:

1. The Airflow `wait_aecb_landing` S3KeySensor has a bounded timeout; if AECB hasn't landed, the sensor does **not** hold the whole DAG hostage indefinitely — the decision-assembly path can still run on the sources that are present (AECB is a LEFT join, not an INNER join).
2. Affected `decision_input` rows have `aecb_credit_score`, `aecb_total_outstanding_aed`, `aecb_report_ref`
= NULL. This is **not** a must-pass failure (the range checks are guarded by "when present"), so the rows are **not** quarantined.

3. `input_completeness_score` drops; if enough rows are affected the completeness and AECB-freshness WARN thresholds fire an **SNS alert** to the on-call, and the day's `dq_score` falls — visible on the `dq_scorecard_daily` mart and `v_dq_scorecard_trend`.
4. The credit policy decides how to treat a thin file (decline / refer / reduced limit). When AECB later arrives, the bookmarked bronze→silver job and the idempotent Delta MERGE **converge** the record on the next run; re-scored decisions get new `decision_ids`, preserving the original decision's snapshot.

The design choice: **degrade gracefully and loudly**, never block the pipeline on one tardy source, and make the degradation a first-class, alertable, auditable signal rather than a silent gap.

3. Compliance & Auditability

How the design supports UAE Central Bank audits

The core compliance primitive is **immutable, tamper-evident decision traceability** (SPEC §7, `glue/silver_layer/write_decision_snapshots.py`, `infra/terraform/s3.tf`):

- For **every** `decision_id`, the exact bytes of all inputs the credit engine saw are frozen into a single JSON object: the resolved `master_customer_id` plus the **verbatim raw** AECB/fraud/AML/profile records, each with its bronze S3 URI and a per-record SHA-256.
- The object is written to a **separate S3 bucket** with Object Lock in COMPLIANCE mode and a 7-year retention — WORM: it cannot be altered or deleted by anyone (including root) for the retention window. A guard bucket policy additionally denies lock-weakening and deletes.
- A `content_sha256` over the whole object is stored in the queryable `decision_input_snapshot` Delta index. **Tamper detection** is then trivial and provable: re-hash the S3 object, compare to the stored hash (the sequence in architecture diagram 5). The S3 object is the **legal record**; the Delta row is the **queryable index**.
- `athena/views/v_decision_audit_trail.sql` gives an auditor a single query from `decision_id` → `snapshot_s3_uri` + `content_sha256` + the resolved identity, and the raw record hashes let them prove the bureau/fraud/AML data used was exactly what the provider sent — full lineage back to bronze.

This satisfies the two things a central-bank examiner asks: **"show me every input to this decision, unaltered" and "prove it hasn't been changed since."**

Retention strategy

Data	Retention	Mechanism
Decision snapshots (legal record)	7 years, immutable	S3 Object Lock COMPLIANCE mode (SNAPSHOT_RETENTION_YEARS, SPEC §11)
Silver/gold Delta tables	Live + time-travel window	Delta <code>_delta_log</code> versions; VACUUM policy tuned to the audit window

Data	Retention	Mechanism
Bronze raw (Parquet/Delta)	Source-of-record window	Lifecycle to cheaper storage class, retained long enough to re-derive silver
All at rest	—	SSE-KMS (single customer-managed key) across S3/RDS/MSK/SNS/Athena
Residency	UAE	Whole stack pinned to me-central-1

GDPR-style deletion vs preserving decision history

This is the hardest tension in the design: ****a data-subject erasure request conflicts directly with the legal obligation to retain the decision record for 7 years.**** UAE PDPL (and GDPR by analogy) both recognise that erasure does **not** override a statutory retention duty — but you still cannot keep unbounded raw PII forever. The Part 1 design leans toward, and the production plan would fully implement:

- **Pseudonymisation by construction.** Downstream everything is keyed on `master_customer_id` (an opaque

UUIDv5), and **gold marts expose no Level 1/2 PII** (SPEC §10) — only natural business keys and aggregates. The analytics/BI surface already contains no directly-identifying data.

- **Crypto-shredding for the "right to be forgotten".** The production answer to erasure-vs-retention is **per-subject encryption keys**: PII in bronze/silver and the snapshot payload is encrypted with a key derived per `master_customer_id`; honouring an erasure request means **destroying that key**. The decision record, hashes, and the *fact* a decision was made survive (satisfying the regulator); the PII becomes unrecoverable ciphertext (satisfying the subject). This threads the needle without breaking Object Lock — you never delete the WORM object, you render its PII unreadable.

- **Tombstone + redaction at the query layer.** A suppression list masks/omits PII for a shredded subject in Athena views, while the immutable snapshot remains as the sealed legal record accessible only under a documented regulatory-access process.

The honest position: ****full physical erasure of a subject inside a 7-year WORM record is legally impossible and undesirable; crypto-shredding is the industry-standard reconciliation****, and it is the first compliance item on the post-launch list because Part 1 ships the pseudonymisation but not yet the per-subject key management.

4. Sharia Compliance Considerations

Where Part 1 stands

Part 1 is intentionally **product-shape-agnostic at the data layer**: `decision_input` carries a `product_code` (PERSONAL_FINANCE | BNPL | CARD_ALT) and generic economics (`requested_amount_aed`, `approved_amount_aed`, `decision_outcome`). Nothing in the pipeline assumes interest-based lending — but

nothing yet *models* the distinctions Islamic finance requires. The current products are, in effect, conventional. Supporting Sharia-compliant products is an **additive data-model extension**, not a rebuild — which is the payoff of keeping the decision record input-agnostic.

Conventional vs Sharia-compliant — what the data model must add

Islamic finance prohibits **riba** (interest), **gharar** (excessive uncertainty), and **haram** activities, and replaces interest with **asset-backed and profit-sharing** structures. The decision platform must therefore capture the *contract structure*, not just an amount and a rate:

- **Contract type dimension.** Extend `product_code` (or add `financing_structure`) to model **Murabaha** (cost-plus sale), **Ijara** (lease-to-own), **Tawarruq** (commodity monetisation), **Musharaka/Mudaraba** (partnership/profit-sharing), **Qard Hassan** (benevolent loan). BNPL, for instance, is typically structured as Murabaha or Tawarruq in an Islamic bank.
- **Profit instead of interest.** Replace/augment an interest field with `**profit_rate / markup_amount / profit_sharing_ratio**` — economically a return, structurally a trade profit or a share of venture profit, and modelled as such for both decisioning and reporting.
- **Underlying asset.** Murabaha and Ijara require a real asset the bank buys and sells/leases — `underlying_asset_type`, `asset_cost_aed`, `ownership_transfer_event`. A decision on an Islamic product needs the asset's existence and valuation as an input.
- **Permissible-purpose & source screening.** Beyond AML/PEP, Sharia products need `purpose_of_finance` screened for halal use and **income-source permissibility** (income from alcohol/gambling/conventional-interest is impermissible). This is naturally an **extension of the existing screening source** (the AML webhook path) with a Sharia-screening result, not a new pipeline.
- **Late-payment treatment.** Penalty interest is prohibited; late fees are typically donated to charity. The model needs a **charity/purification account** and a flag that late charges are non-income — materially different revenue accounting from the conventional `dq_scorecard/portfolio` economics.
- **Governance metadata.** `sharia_board_approval_ref`, product `sharia_certified` flag, and an audit trail of the fatwa/standard (AAOIFI) the product conforms to — which slots directly into the existing immutable-snapshot audit model.

Additional data points needed (summary)

`financing_structure`, `profit_rate / markup_amount / profit_sharing_ratio`,
`underlying_asset_*`,
`ownership_transfer_event`, `purpose_of_finance`, `income_source_permissibility`,
`sharia_screening_status`,
`charity_account_id`, `sharia_board_approval_ref`, `aaofii_standard_ref`. Crucially, the ***decision snapshot*** would freeze the Sharia structure and screening alongside the credit inputs, so a Sharia auditor (in addition to the central bank) can reconstruct that a given approval was structured

compliantly — the same WORM primitive, a richer payload.

5. What I Cut & Why (48-hour scope)

Deprioritised / simplified — and the reasoning

Cut / simplification	Why it was acceptable for launch scope
EMR — ran everything on Glue	Glue meets 10K/day comfortably; the <i>same</i> PySpark runs on EMR later (<code>glue/common/spark_session.py</code>), so this is a config swap, not a rewrite (SPEC §3, <code>glue/README.md</code>)
ML matching / ML credit & fraud features — used deterministic + weighted-Jaro-Winkler and rule-based DQ	Explainable, testable, defensible day-one; ML needs labelled data the platform hasn't yet generated
Merge/unmerge & stewardship UI — shipped the queues as Athena views (<code>v_unresolved_identities</code>)	The data to work the backlog exists and is queryable; the console is UI work with no pipeline risk
Per-subject crypto-shredding — shipped pseudonymisation + PII-free gold	The hard part (keeping PII out of analytics, immutable audit) is done; key-management is a focused follow-up
Real-time streaming for batch/API sources — batch + CDC only	Only the internal profile needs sub-15-min latency (CDC delivers it); AECB/fraud/AML cadences don't justify streaming yet
Full load test to 100K/day, DR/multi-region — single region, sized on paper	Launch is 10K/day; the scale/DR work is explicitly the Day 61–90 plan, gated behind a real load test
Metabase wired end-to-end — Athena + marts ready, BI connection stubbed	The serving contract (gold marts, catalog) is done; connecting the dashboard is low-risk config
Sharia product dimension — product-agnostic decision record, no Islamic structures modelled	Kept the record input-agnostic so the extension is additive (§4), rather than guess the structures under time pressure
CI/CD, secret rotation drills, richer sample data	Secrets model is correct (SSM/Secrets Manager + IAM + KMS, no hardcoded creds); the operational hardening is post-launch

The unifying principle: **ship the load-bearing, hard-to-change primitives correctly** (immutable audit + Object Lock, identity resolution with provenance, the two-tier DQ gate, idempotent Delta MERGE, the pseudonymised model) and **defer the things that are additive or pure operations** (UIs, ML, EMR, DR) where deferral carries no architectural debt.

First post-launch improvements (priority order)

1. **Per-subject crypto-shredding + key management** — closes the erasure-vs-retention gap (§3); highest compliance value.
2. **Stewardship console with a feedback loop** — turns the UNRESOLVED / review backlog into labelled training data and works down the manual queue (§1).
3. **Scale-out entity resolution** — LSH/MinHash blocking + learned linkage (AWS Entity Resolution / Splink) ahead of the 100K/day ramp, plus the EMR migration validated by a real load test.
4. **ML credit & fraud features on a feature store** — built on the now-flowing, snapshotted decision history, with strict train/label temporal separation (no leakage).
5. **Sharia product dimension** — the additive data-model extension in §4, unlocking Islamic-finance products on the same audited pipeline.

Companion to Part 1 (this repository). Architecture: docs/architecture/. Execution plan: docs/execution-plan/. Contracts & thresholds: docs/SPEC.md.